Tab 1

CSE106: Discrete Mathematics
[Fall 2024]

Project Report

Project Title: Random Undirected Graph Generator

Submitted to:

Dr. Mohammad Salah Uddin
Associate Professor
Department of Computer Science & Engineering

Submitted by:

Group- 4 (Scorpion)

Md. Tuhin Alam- (2024-3-60-441)
Sakib Khandakar- (2024-3-60-442)
Al Amin- (2024-3-60-448)
Sajid Ahnaf Sahaf- (2024-3-60-449)

Section- 7
Submission Date- 1st February, 2025

1. Introduction

The program represents graphs using an **adjacency matrix**, computes key properties like the number of **edges** and **vertex degrees**, verifies the **Handshaking Theorem**, and measures **computational time**. By experimenting with different graph sizes, it analyzes **computational complexity** and compares theoretical and practical performance.

2. Features & Functionality

- **Graph Generation:** Creates a random undirected graph using an adjacency matrix.
- **Edge Computation:** Counts the number of edges.
- **Degree Calculation:** Determines the degree of each vertex.
- **Handshaking Theorem Check:** Confirms that the sum of vertex degrees equals twice the number of edges.
- **Performance Measurement:** Tracks execution time in milliseconds.
- **Scalability Tests:** Runs experiments for **n = 1000, 2000, 3000, 4000, and 5000**.

3. Program Output

For each **n**, the program prints:

n = 1000:

```
n = 1000:
  Number of edges = 249849
  Sum of degrees = 499698
  Handshaking theorem : Yes
  Time taken = 15.00 ms
```

n = 3000:

```
n = 3000:
  Number of edges = 2249276
  Sum of degrees = 4498552
  Handshaking theorem : Yes
  Time taken = 233.00 ms
```

n = 5000:

```
n = 5000:
  Number of edges = 6248844
  Sum of degrees = 12497688
  Handshaking theorem : Yes
  Time taken = 417.00 ms
```

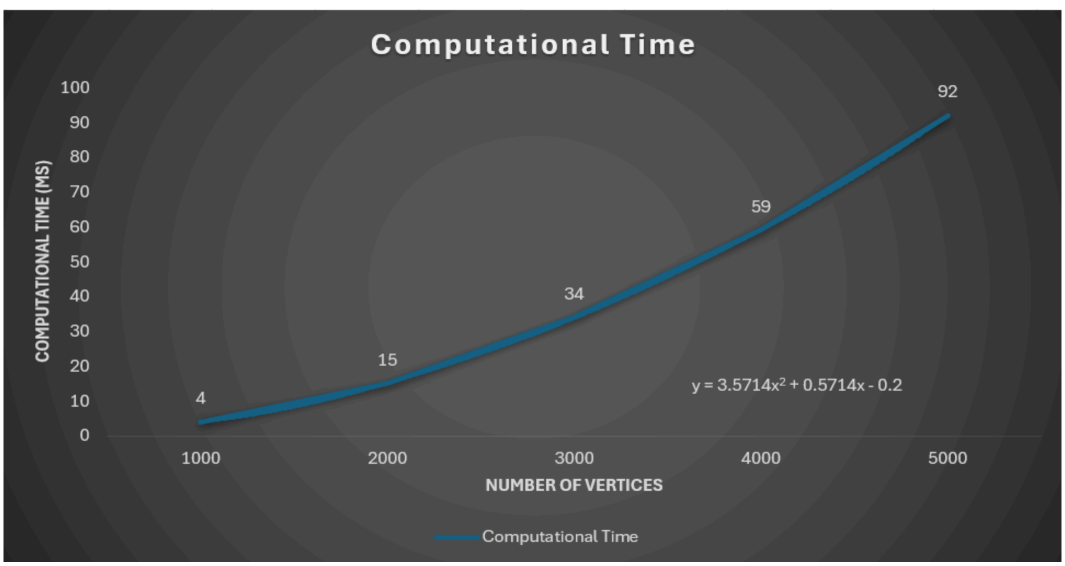
4. Results & Analysis

4.1 Theoretical Complexity

Statement	Big-O Notation
`for (int i = 0; i < n; i++) { for (int j = i + 1; j < n; j++) { edge = rand() % 2; if (edge) { edge_count++; degree[i]++; degree[j]++; } } }`	$O(n^2)$

Algorithm is expected to have a **$O(n^2)$ time complexity** due to the nested loops in adjacency matrix generation and edge counting.

4.2 Experimental Findings



- A **polynomial trendline** was applied to the data.
- Results confirmed that the algorithm follows the expected **$O(n^2)$ complexity**.

5. Conclusion

This project successfully demonstrated the random generation and analysis of undirected graphs using an adjacency matrix. The results aligned well with theoretical expectations, confirming the quadratic time complexity of the algorithm.

The data was analyzed using **Microsoft Excel**, providing real-world insights into computational performance. This project not only strengthens our understanding of graph theory but also highlights the importance of algorithmic efficiency.

Repository: https://github.com/Genio22/Graph-Analysis/blob/int/graph_generator.c